Remediation Architecture for TASK.md: Eliminating Ghost Canon via Pointer-First Governance

1. Executive Summary: The Imperative of Canonical Purity

The operational stability of the Stela repository architecture is predicated on a single, inviolable concept: the **Policy of Truth (PoT)**. As the constitutional document governing the platform, PoT.md defines the physical laws, staffing protocols, and jurisdictional boundaries that enable autonomous and semi-autonomous agents to function without hallucination or drift.[1] However, a forensic analysis of the repository's current state reveals a critical vulnerability in the "Active Work Surface," specifically within TASK.md. This vulnerability is identified as **"Ghost Canon"**—the presence of static, duplicated, or summarized governance text within the operational dashboard that competes with, rather than points to, the authoritative source of truth.

Ghost Canon creates a "Soft Law" environment where agents prioritize immediate, often stale instructions found in their active context (TASK.md) over the immutable axioms defined in PoT.md or the dynamic constraints of the repository state. This redundancy violates the **Single Source of Truth (SSOT)** axiom defined in PoT.md Section 1.2, which mandates that "one canonical file per domain" exists and "other mentions are pointers".[1] When TASK.md contains narrative instructions such as "Do not edit main directly" or "Run required gates," it effectively forks the governance model, creating a parallel jurisdiction that is susceptible to entropy. If the primary law changes, the ghost copy remains, leading to "Ambiguity"—a defined System Failure State requiring an immediate cessation of work.[1]

This report presents a comprehensive remediation proposal to transition TASK.md from a narrative instruction sheet to a rigorous **Pointer-First Dashboard**. By systematically excising all static governance prose and replacing it with immutable pointers to PoT.md, docs/MANUAL.md, and ops/lib/manifests/CONTEXT.md, the architecture eliminates the vector for governance drift. The analysis synthesizes evidence from the repository's **State of Play (SoP)** ledger—specifically DPs 0033, 0038, 0039, and 0051—to demonstrate that "Pointer-First" is not merely a stylistic preference but a proven architectural necessity for maintaining "Pattern Density" and "Truth Topology" in high-context AI environments.[1]

The proposed architecture redefines TASK.md as a stateless connector: it holds the current **Session State** (injected dynamically), references the **Law** (via pointers), and logs the **Intent** (the Dispatch Packet). It contains no definitions, no rules, and no explanations—only

coordinates. This shift ensures that the agent's context window is optimized for execution logic rather than administrative boilerplate, thereby enforcing the "Context Hazard" axiom and adhering strictly to the "Read-in Order" defined in PoT.md.[1]

---

2. The Theoretical Framework of Ghost Canon

2.1 Defining the Governance Hazard

In the Stela governance model, "Truth" is not a metaphysical abstraction but a "primitive predicate governed by specific rules".[1] The system operates on the axiom that "everything the system proves is actually true." Consequently, any divergence between the defined canon and the repository state is a "Drift" failure state.[1]

Ghost Canon is defined as any text within a secondary surface (like TASK.md or a prompt) that attempts to define, summarize, or enforce a rule that belongs to a primary surface (PoT.md). This creates a hazard because the "Ghost" version lacks the update mechanisms of the "Canon" version.

- **Canon:** "Do not push to main." (Located in PoT.md, subject to version control and PR review).
- **Ghost:** "Do not edit main directly." (Located in TASK.md template, potentially persisting across sessions if not scrubbed).

While semantically similar, the divergence creates a cognitive friction for the agent. If PoT.md is updated to allow "Emergency Hotfixes on Main," but TASK.md retains the static prohibition, the agent faces a logic conflict. Under PoT.md Section 2, "Ambiguity" mandates a **STOP WORK** response.[1] Ghost Canon, therefore, is a dormancy virus; it remains benign until the underlying law changes, at which point it causes system paralysis.

2.2 The "Soft Law" Environment

The presence of governance instructions in TASK.md establishes a hierarchy of "Soft Law." Because TASK.md is the "Active Work Surface" [1] and is often the most recently modified file in the context window, Large Language Models (LLMs) assign it disproportionate attention weight.

If TASK.md contains a summarized list of "Do's and Don'ts," the agent treats this summary as the *effective* constitution, ignoring the nuanced clauses in PoT.md. This effectively nullifies the "Read-in Order" mandated by PoT.md Section 1.3 [1], which places PoT.md first and TASK.md third. The remediation must strip TASK.md of this unearned authority to restore the constitutional hierarchy.

2.3 The Context Hazard Axiom and Token Economy

The "Context Hazard" axiom explicitly forbids including JIT (Just-In-Time) documentation—such as docs/library/agents—in the global context manifest to prevent pollution.[1] This principle implies a conservation of attention.

Every token of static instruction in TASK.md (e.g., "Run verify.sh to check the tree") consumes context window space that could be used for the Dispatch Packet's specific execution logic. Furthermore, static instructions decrease the **Pattern Density** of the prompt. If 50% of the active context is repetitive boilerplate, the agent's ability to detect novel patterns or "Low-Frequency Role Emergence" [1] is degraded. The remediation aims to maximize Pattern Density by reducing TASK.md to high-entropy signal (current state, current intent) and pointers to low-entropy static knowledge.

---

3. Architectural Analysis of the Stela Platform

To construct a valid remediation, we must first dissect the existing constitutional architecture as revealed in the repository dump and verify the specific roles of the governance surfaces.

3.1 The Constitution: PoT.md

The **Policy of Truth (PoT)** is the absolute authority. Its jurisdiction is total, covering ops/, docs/, projects/, and tools/.[1] It establishes the fundamental "Physical Laws" of the repository.

3.1.1 The Source of Truth Read-in Order

A critical addition in **DP-OPS-0038B** [1] was the explicit definition of the Read-in Order:

1. **PoT.md**: The Constitution.
2. **SoP.md**: The History.
3. **TASK.md**: The Contract.
4. **docs/MAP.md**: The Terrain.

This hierarchy is non-negotiable. TASK.md exists *under* PoT.md. Therefore, TASK.md cannot define behavioral standards; it can only reference the standards defined in PoT.md. The remediation proposal enforces this by removing the "Behavioral Logic" summary often found in legacy TASK.md templates and replacing it with a pointer to PoT.md Section 4.2.

3.1.2 System Failure States

PoT.md defines four specific triggers for a system halt [1]:

- **Canon Drift:** Divergence between PoT and repo state.
- **Context Hazard:** Including library docs in the global manifest.
- **Dirty State:** Uncommitted changes in main (explicitly exempting TASK.md).
- **Ambiguity:** Instructions capable of multiple interpretations.

The "Dirty State" exemption for TASK.md is crucial. It confirms that TASK.md is a mutable, living surface, unlike the immutable trunk. This mutability makes it a dangerous place to store permanent laws (Ghost Canon) because it is inherently transient and scratchpad-like.

3.2 The Ledger: SoP.md

The **State of Play (SoP)** provides the historical context for *why* the system is architected this way. A forensic review of recent entries [1] reveals a clear, converging trajectory toward "Pointer-First" governance.

Table 1: Evolutionary Trajectory of Pointer-First Architecture

Date	DP-ID	Title	Architectural Contribution
2026-02-07	DP-OPS-0033	Registry Consolidation	Decoupled Data (Registry) from Pointers (Index), establishing the pattern of separation.[1]
2026-02-09	DP-OPS-0038	Pointer-First Constitution	Replaced TRUTH.md with PoT.md and refactored TASK.md to a dashboard schema.[1]
2026-02-10	DP-OPS-0042	Agent Certification	Hardened Agent files to be pointer-first, using "Pattern Density" to detect roles.[1]
2026-02-11	DP-OPS-0046	Constitution Refinement	Explicitly removed duplicated canon text from TASK.md.[1]
2026-02-12	DP-OPS-0051	System	Eliminated TASK.md

		Remediation	ghost-canon drift vectors by switching the Freshness Gate to pointer-first governance.[1]

The remediation proposal is the logical endpoint of this trajectory. It formalizes the work begun in DP-OPS-0038 and refined in DP-OPS-0051, creating a locked-down template that resists regression.

3.3 The Lens: ops/lib/manifests/CONTEXT.md

The CONTEXT.md manifest is the enforcement mechanism for context purity. As defined in the documentation: "Context is not memory; it is a file".[1] The manifest strictly defines what an agent *is*—the sum of the files listed therein.

The manifest is divided into:

- **Small Bundle:** PoT.md, SoP.md, TASK.md, llms.txt.
- **Full Bundle:** Adds MANUAL.md, MAP.md, GOVERNANCE.md, and Registries.

The remediation proposal mandates that TASK.md must point *to* this manifest as the definition of required context. Legacy TASK.md templates often contained a checklist: "Ensure you have PoT, SoP, and MAP loaded." This is Ghost Canon. If the manifest changes (e.g., adding PROMPTS.md), the TASK.md checklist becomes stale. The correct pattern is: "Context Manifest: ops/lib/manifests/CONTEXT.md."

---

4. Forensic Audit of Dispatch Packet Evolution

By analyzing the provided Dispatch Packets, we can identify specific failure modes of the legacy TASK.md structure and validate the efficacy of the pointer-first approach.

4.1 DP-OPS-0033: The Registry Precedent

In **DP-OPS-0033** [1], the system suffered from "double-bookkeeping." Registry data existed in both docs/library/INDEX.md and docs/ops/registry/. This caused drift where one file would be updated and the other forgotten.

- **Remediation:** INDEX.md was stripped of all data tables and converted into a navigation surface containing only links (pointers).
- **Relevance:** This proves the core thesis: Data and Logic must live in exactly one place (SSOT). TASK.md currently suffers from the same "double-bookkeeping" regarding

governance rules.

4.2 DP-OPS-0038: The Constitutional Shift

DP-OPS-0038 [1] replaced the non-existent TRUTH.md with PoT.md. It introduced the "Mandatory Closing Block" in TASK.md.

- **Failure Mode:** The DP noted that TASK.md relied on "narrative instructions" and "Ghost Canon references."
- **Corrective Pattern:** The DP refactored TASK.md to include "Session State" and "Logic Pointers." The remediation proposal builds on this by making those pointers *exclusive*—removing any accompanying explanatory text like "Follow the filing doctrine defined in PoT."

4.3 DP-OPS-0039: Quantitative Rigor

DP-OPS-0039 [1] enforced explicit Closeout pointers in B-TASK execution logic.

- **Insight:** Narrative steps like "Close the task" were insufficient. The system required Complete Closeout per TASK.md Section 4.
- **Implication:** This proves that *pointers are more executable than prose*. An instruction to "Run cleanup" is ambiguous. An instruction to "Execute docs/MANUAL.md Closeout Cycle" is deterministic because it refers to a specific, version-controlled procedure.

4.4 DP-OPS-0051: The Drift Vector Elimination

The entry for **DP-OPS-0051** in SoP.md [1] explicitly claims to have "Eliminated TASK.md ghost-canon drift vectors."

- **Mechanism:** Switching the Freshness Gate to "pointer-first governance" and updating ops/bin/prune to strip legacy static blocks.
- **Significance:** This confirms that static blocks of text (e.g., lists of files to check) are "drift vectors." If the file list changes in ops/bin/open but not in the TASK.md static text, the system drifts. The remediation proposal codifies the rule that TASK.md must never contain static lists of dynamic artifacts.

4.5 DP-OPS-0037: The Import/Port Workflow

DP-OPS-0037 [1] demonstrated the danger of "Porcelain inputs" (untracked files). It required a strict classification of "imports" before promotion.

- **Relevance:** TASK.md is often treated as porcelain (dirty state allowed). This makes it highly volatile. Relying on a volatile file for immutable governance rules (like "Do not edit main") is architecturally unsound. Governance must live in PoT.md (Main/Canon), referenced by TASK.md (Porcelain/Work).

---

5. The Remediation Proposal: Pointer-First TASK.md Schema

This section defines the rigorous, drift-proof schema for TASK.md. The guiding principle is **Indirection**: TASK.md contains the address of the rule, never the rule itself.

5.1 Section 1: Session State (The Anchor)

Legacy versions of TASK.md often contained hardcoded checks for branch naming conventions or manually updated HEAD hashes. The new schema delegates this entirely to the machine, specifically the ops/bin/open binary.[1]

Proposed Schema:

STELA TASK DASHBOARD (LIVING SURFACE)

Status: ACTIVE

Owner: Integrator

Last Updated: YYYY-MM-DD

1. Session State (The Anchor)

Pointer: Session context output (generated by ops/bin/open)

Active Branch: main (Must match session context output)

Base HEAD: (Must match session context output)

Context Manifest: ops/lib/manifests/CONTEXT.md (Checked by tools/lint/context.sh)

Rationale:

- **Dynamic Source:** The "Pointer" references ops/bin/open output. The TASK.md does not need to explain *how* to generate session context; it simply points to the artifact that contains it.
- **Validation:** It explicitly points to ops/lib/manifests/CONTEXT.md as the arbiter of context completeness, monitored by tools/lint/context.sh. This eliminates the need for TASK.md to list "Required Files," which was a major source of Ghost Canon drift.

5.2 Section 2: Logic Pointers (The Law)

This is the primary remediation surface for Ghost Canon. Previously, TASK.md might list "Do not use contractions," "Update SoP," or "Follow Filing Doctrine." These are removed in favor of strict pointers to the files that enforce them.

Proposed Schema:

2. Logic Pointers (The Law)

Primary Constraint: PoT.md (Policy of Truth).

2.1 Governance Pointers

Jurisdiction: PoT.md Section 3.

Git Authority: PoT.md Section 4.1.

Behavioral Standard: PoT.md Section 4.2.

Staffing Protocol: PoT.md Section 4.1.

2.2 Execution Pointers (The Toolchain)

Linguistic Precision: tools/lint/style.sh.

Structure Verification: tools/verify.sh.

Context Hygiene: tools/lint/context.sh.

Truth Integrity: tools/lint/truth.sh.

DP Validation: tools/lint/dp.sh.

llms Integrity: tools/lint/llms.sh.

Rationale:

- **Governance Pointers:** Instead of summarizing the "Staffing Protocol" (e.g., "Operators commit code"), the dashboard points to PoT.md Section 4.1. If PoT.md changes (e.g., giving Contractors commit access to specific branches), TASK.md remains accurate without editing.
- **Execution Pointers:** Instead of listing rules like "No contractions," it points to tools/lint/style.sh. The *code* is the law, not the text. This aligns with the "Enforcement Pointers" concept in docs/GOVERNANCE.md [1], creating a direct link between the dashboard and the verification binary.

5.3 Section 3: Current Dispatch Packet (The Intent)

This section hosts the JIT instructions (the DP). To prevent Ghost Canon here, the template must be minimal and rely on external definitions for "Freshness."

Proposed Schema (Freshness Gate):

3. Current Dispatch Packet (DP)

DP-XXXX: (Template)

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/dp-ops-XXXX-YYYY-MM-DD

Base HEAD: (Must match session context output)

Preconditions:

- No commits on main.
- Working tree must be clean before execution begins.
- If Base HEAD changes, regenerate session artifacts from canonical tools before proceeding.

Canonical pointers (no duplicated canon text in TASK.md):

- Behavioral logic and drift authority: PoT.md.
- Closeout mechanics and command canon: docs/MANUAL.md.
- Gate canon: tools/lint/context.sh and tools/verify.sh.
- Artifact and scrub canon: ops/bin/open, ops/bin/dump, and ops/bin/prune.

Rationale:

- **Explicit Exclusion:** The line "(no duplicated canon text in TASK.md)" acts as a meta-rule for the agent, explicitly forbidding the generation of summaries.
- **Pointer Consolidation:** It groups pointers by domain (Behavior, Closeout, Gate, Artifact). This forces the agent to read docs/MANUAL.md [1] for the "Closeout Cycle" rather than relying on a potentially outdated summary in TASK.md.

5.4 Section 3.4: Execution Plan (A-E Canon)

The remediation enforces the "A-E Canon" structure (State, Request, Changelog, Patch, Receipt), which is validated by tools/lint/dp.sh.[1] The most critical change is in the Receipt section.

Proposed Schema (Receipt):

3.4.5 Receipt (Proofs to collect)

- Artifacts captured: OPEN, OPEN-PORCELAIN (or none), dump payload, dump manifest, DP-OPS-XXXX-RESULTS.md.
- Gates run: bash tools/lint/context.sh, bash tools/lint/style.sh, bash tools/lint/truth.sh, bash tools/lint/dp.sh TASK.md, bash tools/verify.sh.

Rationale:

- **Deterministic Proofs:** The list of "Gates run" mirrors the repo-gates.yml workflow.[1] By hardcoding the *command lines* rather than descriptions (e.g., "Run lints"), we ensure exact reproducibility. This prevents the "Ambiguity" failure state defined in PoT.md.

5.5 Section 4: Closeout (The Handoff)

This section historically accumulated the most Ghost Canon (e.g., "Remember to prune," "Don't forget to squash," "Update the map"). The proposal strips this entirely, replacing it with a pointer to the "Closeout Cycle" in the Manual.

Proposed Schema:

4. Closeout (Mandatory)

- Execute docs/MANUAL.md Closeout Cycle in order (Verify, Harvest, Refresh, Log, Prune).
- Update SoP.md with a DP entry at completion, including objective summary and verification commands run.
- Run Routing Closeout scrub at the end of the closeout sequence:./ops/bin/prune --dp=DP-OPS-XXXX --scrub.
- Ensure the next session begins with refreshed session context output and matching state capture artifacts.

Mandatory Closing Block

Varied Wording provision: Each entry must use meaningfully distinct wording; copy or minor tense changes are not acceptable.

Entry 4 must differ from Entry 1.

Primary Commit Header (plaintext)

(Write to DP-OPS-XXXX-RESULTS.md; do not pre-fill in TASK.md)

Pull Request Title (plaintext)

(Write to DP-OPS-XXXX-RESULTS.md; do not pre-fill in TASK.md)

Pull Request Description (markdown)

(Write to DP-OPS-XXXX-RESULTS.md; do not pre-fill in TASK.md)

Final Squash Stub (plaintext) (Must differ from #1)

(Write to DP-OPS-XXXX-RESULTS.md; do not pre-fill in TASK.md)

Extended Technical Manifest (plaintext)

(Write to DP-OPS-XXXX-RESULTS.md; do not pre-fill in TASK.md)

Review Conversation Starter (markdown)

(Write to DP-OPS-XXXX-RESULTS.md; do not pre-fill in TASK.md)

Rationale:

- **Single Pointer:** The instruction "Execute docs/MANUAL.md Closeout Cycle" replaces paragraphs of text. docs/MANUAL.md [1] contains the authoritative definition of the Closeout Cycle (Verify, Harvest, Refresh, Log, Prune). This centralizes the process logic so that if the cycle changes (e.g., adding a new llms refresh step), TASK.md does not need to be updated.
- **Results Redirection:** Instructions explicitly state "Write to DP-OPS-XXXX-RESULTS.md; do not pre-fill in TASK.md." This prevents TASK.md from becoming a storage dump, keeping it a clean dashboard for the next session. This aligns with the ops/bin/prune logic [1], which scrubs TASK.md back to its template state.

---

6. Operationalizing via Context Manifests

The TASK.md remediation relies on the ops/lib/manifests/CONTEXT.md file being the absolute source of truth for what an agent "knows."

6.1 The Manifest as the Gating Mechanism

The tools/lint/context.sh script [1] verifies that every file listed in ops/lib/manifests/CONTEXT.md exists. This creates a closed verification loop:

1. TASK.md points to CONTEXT.md as the "Context Manifest."
2. CONTEXT.md lists PoT.md, SoP.md, and docs/MANUAL.md.
3. tools/lint/context.sh verifies CONTEXT.md against the file system.
4. Therefore, TASK.md validates the presence of the governance layer indirectly but

robustly, without needing to list every governance file itself.

6.2 Preventing Context Hazards

The remediation proposal explicitly reinforces the "Context Hazard" check. TASK.md must **not** contain deep links into docs/library/ (agents, tasks, skills) unless they are part of the active DP execution plan.

- **Logic:** docs/library/ content is JIT (Just-In-Time). Loading it globally pollutes the context.[1]
- **Enforcement:** tools/lint/context.sh scans for "Context Hazard" patterns (docs/library/(agents|tasks|skills)) and fails if they appear in the manifest. TASK.md complies by only referencing the *Registry* or the *Index* via pointers, never the raw library files in its static blocks.

Table 2: Comparison of Old vs. New TASK.md Schemas

Feature	Legacy TASK.md (Ghost Canon)	Pointer-First TASK.md (Remediated)	Source of Truth
Governance	"Do not edit main."	Jurisdiction: PoT.md Section 3	PoT.md
Branching	"Work on work/* branches."	Staffing Protocol: PoT.md Section 4.1	PoT.md
Context	List of required files.	Context Manifest: ops/lib/manifests/CONTEXT.md	CONTEXT.md
Closeout	Step-by-step prose instructions.	Execute docs/MANUAL.md Closeout Cycle	MANUAL.md
Verification	"Run lints."	bash tools/lint/style.sh, etc.	tools/lint/*.sh
Drift Risk	**High** (Template deviates from PoT).	**Zero** (Template points to PoT).	N/A

7. Implementation Strategy

To transition from the current state to the proposed Pointer-First architecture, the following execution plan is required. This mirrors the structure of a standard Stela Dispatch Packet and accounts for the toolchain dependencies identified in the research.

7.1 Phase 1: The Template Scrub

Objective: Reset TASK.md to the zero-state defined in Section 5.

- **Action:** Edit TASK.md to remove all "Do/Do Not" lists, all "Role Contract" descriptions, and all "Freshness Gate" detailed checklists.
- **Replacement:** Insert the high-level pointers to PoT.md and docs/MANUAL.md.
- **Automation:** Update ops/bin/prune [1] to recognize the new TASK.md structure. The scrub_task function in ops/bin/prune currently has hardcoded awk patterns (e.g., /^### 3\.1 /). These must be updated to match the new "Logic Pointers" section headers, or the prune script will fail to reset the file correctly.

7.2 Phase 2: Toolchain Alignment

Objective: Ensure ops/bin/open and tools/lint/dp.sh support the new schema.

- **ops/bin/open:** This binary injects session state.[1] It currently uses awk to find Active Branch: and Base HEAD:. It must be updated to target the specific Session State block in the new TASK.md and ensure it does *not* inject redundant "Role Contract" text into the OPEN artifact if TASK.md is now the primary pointer source.
- **tools/lint/dp.sh:** This linter validates the DP structure.[1] It must be refactored to enforce the presence of the "Logic Pointers" section and the "Canonical pointers" block. Specifically, the check_task_subsections function in dp.sh must be updated to look for ## 2. Logic Pointers instead of legacy headers. It should also include a negative check (grep) that fails if it detects Ghost Canon phrases like "Do not edit main" directly in the file.

7.3 Phase 3: Manual and Map Synchronization

Objective: Ensure the pointer targets are robust.

- **docs/MANUAL.md:** The "Closeout Cycle" section must be verified to be fully self-contained and explicit, as TASK.md now delegates all authority to it.
- **docs/MAP.md:** Verify it correctly indexes the "Terrain" so the TASK.md pointer to it is meaningful.

8. Third-Order Insights: The Impact on Agent Cognition

Transitioning TASK.md to a Pointer-First architecture yields systemic benefits beyond simple cleanup. These insights are derived from analyzing the "Pattern Density" concepts introduced in **DP-OPS-0042**.[1]

8.1 Pattern Density and Low-Frequency Emergence

By removing static noise from TASK.md, we increase the **Pattern Density** of the active Dispatch Packet.

- **Mechanism:** LLMs have a finite attention mechanism. If 50% of the TASK.md context is static boilerplate ("Do not edit main," "Run lints"), the model's attention is diluted across low-value tokens.
- **Result:** By reducing this to < 5% of tokens (via pointers), the model focuses intensely on the *unique* aspects of the current DP (the Objective, the Execution Plan). This increases the likelihood of detecting "Low-Frequency Role Emergence" [1]—subtle architectural needs that would otherwise be drowned out by governance noise.

8.2 Flattening the Truth Topology

The proposal enforces a "Flat Truth" topology.

- **Current State:** Hierarchical/Fractured. Truth lives in PoT, SoP, and scattered copies in TASK.md. This requires the agent to reconcile conflicts (e.g., "PoT says X, but TASK says Y").
- **Proposed State:** Flat. PoT is the *only* mountain. TASK.md is a map reference to that mountain.
- **Implication:** This makes the system "Anti-Fragile" to governance changes. If PoT.md changes (e.g., adding a new "Security Review" role), TASK.md inherits this immediately because it points to PoT.md Section 4.1, rather than listing roles explicitly.

8.3 The "Headless" Dashboard

The remediation moves TASK.md toward a "Headless" state—a file that is machine-readable and human-readable but contains minimal *interpretive* text. This paves the way for higher-order automation where an "Integrator" agent can parse TASK.md strictly as a JSON-like structure of pointers and state, without needing to parse natural language instructions. This aligns with the "Linguistic Precision" and "No Contractions" standards of PoT.md [1], treating the dashboard as configuration rather than conversation.

---

9. Conclusion

The "Ghost Canon" phenomenon is a structural debt that threatens the determinism of the Stela platform. By allowing TASK.md to carry duplicate governance logic, the system invites drift and ambiguity, triggering the "System Failure States" defined in the Constitution. The remediation proposal presented here—strictly enforcing a **Pointer-First** architecture—is not merely a cleanup; it is a constitutional realignment.

By stripping TASK.md to its bare essentials—State, Pointers, Intent, and Log—and delegating all authority to PoT.md and ops/lib/manifests/CONTEXT.md, we achieve a system where **Action** is decoupled from **Definition**. The TASK.md file becomes a pure instrument of execution, validated by the rigorously defined "Context Manifest" and policed by the "Policy of Truth." This creates a repository state that is self-healing, audit-ready, and functionally unambiguous.

Recommended Immediate Actions (Next Operator Moves)

1. **Draft DP-OPS-00XX:** "Pointer-First TASK.md Remediation."
2. **Execute Phase 1:** Scrub the TASK.md template to the schema defined in Section 5.
3. **Execute Phase 2:** Refactor tools/lint/dp.sh to enforce the new schema and ban Ghost Canon phrases.
4. **Execute Phase 3:** Update docs/MANUAL.md to ensure the "Closeout Cycle" is fully self-contained.
5. **Verify:** Run ops/bin/dump and tools/verify.sh to confirm the new baseline is drift-free.

This approach guarantees that TASK.md remains the "Active Work Surface" without usurping the "Constitutional Authority," preserving the delicate balance of the Stela Staffing Protocol.

Works cited

1. dump-full-main-088b6100.txt